ctc_asr_text_correct

애플리케이션 내에서 바로 동작하는 한국어 음성 인식(fine-tuned Wav2Vec2-CTC) + Gemini 전사 교정 라이브러리, 준실시간 스트리밍 지원.

GitHub: github.com/Aditi1612/ctc_asr_text_correct_library · 모델: huggingface.co/fluffy16/ctc-asr-ko (승인제) · v0.1.0

개요

한국어 ASR 모델을 **애플리케이션 프로세스 안에서 직접** 실행하는 Python 라이브러리입니다. HTTP 서버도, API 엔드포인트 호출도 필요 없습니다. 모델을 한 번만 로드한 뒤 함수를 호출하면 됩니다. 전사 결과는 이어서 Google Gemini로 전달되어 잘못 인식된 음절, 띄어쓰기, 문법이 교정됩니다.

```
오디오 → Wav2Vec2-CTC (파인튜닝, 한국어) → 전사 → Gemini → corrected_text
```

- **단건(one-shot)**: 오디오 파일이나 파형을 넘기면 원본 전사와 교정된 텍스트를 함께 돌려줍니다.
- **스트리밍**: 마이크 프레임을 계속 밀어 넣으면, 화자가 말을 멈출 때마다 방금 말한 발화의 교정된 전사를 반환합니다.
- **한쪽만 사용**: 음성 인식만(Gemini 키 불필요), 또는 교정만(모델 다운로드 불필요) 사용할 수도 있습니다.

음성 인식 경로(오디오 정규화, CTC 디코딩, 자모 정규화, 한글 조합)는 운영 중인 CTC API와 동일하므로 전사 결과도 같습니다. 교정 단계는 운영 파이프라인과 같은 프롬프트와 Gemini 키 순환을 사용합니다. 모델은 약 1.2 GB로 작고, CPU에서도 충분히 빠르게 동작합니다.

시작하기 전에: 두 가지 접근 권한

항목	이유	받는 방법
모델에 대한 Hugging Face 접근 권한	모델 저장소는 **승인제(gated)**입니다. 승인된 계정만 가중치를 다운로드할 수 있습니다.	1. huggingface.co에 로그인한 뒤 fluffy16/ctc-asr-ko를 엽니다. 2. **Request access**를 클릭합니다. 관리자가 이미 계정을 추가해 둔 경우에는 이 단계를 건너뜁니다. 3. 승인을 기다립니다. 4. huggingface.co/settings/tokens에서 **Read** 토큰을 만듭니다.
Gemini API 키	교정 단계에서 Google Gemini API를 호출합니다.	aistudio.google.com/apikey에서 키를 만들거나(무료 등급 있음), 관리자에게 요청하세요.

> HF 토큰과 Gemini 키는 비밀번호처럼 다루세요. git에 커밋하거나 공유 문서에 붙여 넣지 말고, 아래처럼 환경 변수로 설정하세요.

요구 사항

구성 요소	설명
Python	3.9 이상
`torch`, `torchaudio`	사용하는 환경(CUDA 또는 CPU)에 맞는 빌드를 먼저 설치하세요. 용량이 크고 하드웨어에 종속적입니다.
GPU	선택 사항. CUDA에서는 클립당 약 25 ms, CPU에서도 오디오 1초당 약 0.04 초로 충분히 동작합니다.
디스크	모델 약 1.3 GB. 최초 1회 다운로드 후 `~/.cache/huggingface`에 캐시됩니다.
네트워크	최초 모델 다운로드와 매번의 Gemini 교정 호출에 필요합니다.

설치

```
# 1. 하드웨어에 맞는 torch 설치 (예: CUDA 12.1)
pip install torch torchaudio --index-url https://download.pytorch.org/whl/cu121
#    ...또는 CPU 전용:
pip install torch torchaudio --index-url https://download.pytorch.org/whl/cpu

# 2. 라이브러리 설치
pip install "ctc-asr-text-correct[hub] @ git+https://github.com/Aditi1612/
ctc_asr_text_correct_library.git"

# 3. 인증 정보 (셸 프로필이나 앱이 읽는 .env에 넣어 두세요)
huggingface-cli login                    # HF Read 토큰 붙여 넣기, 또는: export HF_TOKEN=hf_...
export GEMINI_API_KEY=your-gemini-key
export CTC_HF_REPO=fluffy16/ctc-asr-ko     # 선택: 코드에서 hf_repo= 생략 가능
```

선택적 extras:

Extra	추가 기능
`[hub]`	Hugging Face Hub에서 모델 다운로드 (원래 서버 밖에서는 필수)
`[vad]`	`webrtcvad` — 내장 에너지 기반 방식보다 정확한 음성 구간 검출
`[mic]`	`sounddevice` — 마이크 데모 실행 시에만 필요

빠른 시작 — 음성 인식 + 교정

```
from ctc_asr_text_correct import ASRCorrector

pipe = ASRCorrector(hf_repo="fluffy16/ctc-asr-ko")   # 최초 실행 시 약 1.2 GB 다운로드, 이후 캐시

r = pipe.process("clip.wav")
r.transcript          # '카촉틀과 토이띠차 내뒤에 안자쩌요'    음성 인식 원본 출력
r.corrected_text      # 같은 문장을 Gemini가 교정한 결과
r.raw_transcription   # 한글 조합 전, 디코딩되지 않은 CTC 자모 출력
r.correction_ok       # True    Gemini가 실패해 대체 처리를 썼으면 False
r.timings             # {'stt': 0.02, 'text_correction': ..., 'total': ...}
r.to_dict()           # 위 내용 전체를 일반 dict로 (JSON 직렬화 가능)
```

`process()` 는 입력으로 파일 경로, 인코딩된 바이트(`wav/flac/ogg` , ffmpeg가 있으면 `mp3/m4a` 도 가능), 또는 `input_sr=` 과 함께 전달한 numpy / torch 파형을 받습니다.

인자	기본값	의미
`reference_text`	—	정답 전사. 전달하면 `r.cer` (문자 오류율)이 채워짐
`prompt`	내장 프롬프트	이번 호출의 교정 지시문을 대체

한쪽만 사용하기

```
from ctc_asr_text_correct import ASRCorrector, Transcriber, GeminiCorrector

# 음성 인식만: Gemini 키 불필요
asr = Transcriber(hf_repo="fluffy16/ctc-asr-ko")
asr.transcribe("clip.wav")                    # -> str
asr.transcribe("clip.wav", detailed=True)     # -> raw_transcription / cer 포함 dict

# 교정만: 모델 다운로드 없이 Gemini 키만 필요
fix = GeminiCorrector()
fix.correct("은 변원 앞에 있어요 약을 하우레 두 씩 드세요")
# -> '병원 앞에 있어요. 약을 하루에 두 시 씩 드세요.'

# 이미 만든 객체를 조합
pipe = ASRCorrector(transcriber=asr, corrector=fix)
```

스트리밍 (준실시간)

애플리케이션이 어디서든 오디오를 캡처해 길이에 상관없이 mono float32 청크로 밀어 넣습니다. 화자가 말을 멈추면 그때까지 버퍼링된 발화가 전사·교정되어 반환됩니다. 부분 결과(partial)는 없으며, 발화마다 완성된 전사 하나가 제공됩니다.

```
from ctc_asr_text_correct import ASRCorrector

pipe = ASRCorrector(hf_repo="fluffy16/ctc-asr-ko")
stream = pipe.stream(on_segment=lambda s: print(s.text, "->", s.corrected_text))

for chunk in your_mic_source():                       # 예: 20~100 ms 블록, 샘플레이트 무관
    for seg in stream.accept(chunk, input_sr=48000):  # 필요 시 내부에서 리샘플링
        handle(seg.corrected_text, seg.start_time, seg.end_time)

stream.flush()                                        # 스트림 종료 시 남은 발화를 마무리 처리
```

- `accept()` 는 확정된 `Segment` 목록을 반환합니다. 보통은 비어 있고, 발화가 막 끝났을 때 하나가 담깁니다. 각 세그먼트에는 `.text` (음성 인식 원본), `.corrected_text`, `.start_time`, `.end_time` 이 있습니다.
- `pipe.stream(correct=False)` 로 만들면 Gemini를 건너뛰며, `corrected_text` 는 `None` 이 됩니다.
- 발화가 끝나면 음성 인식과 Gemini 호출이 실행되는 동안 `accept()` 가 블로킹됩니다. 음성 인식은 GPU에서 수십 ms에 불과하므로 대기 시간은 대부분 Gemini 왕복 시간(약 0.5~1초)입니다. 오디오 캡처 스레드가 멈추면 안 된다면, 큐를 두고 별도 워커 스레드에서 `accept()` 를 호출하세요.
- 실행 가능한 마이크 데모: `examples/stream_mic.py`

`pipe.stream(...)` 튜닝

인자	기본값	의미
`end_silence_ms`	600	발화를 끝내는 침묵 길이 — 체감 지연 시간과 거의 같음
`min_speech_ms`	250	이보다 짧은 발화는 잡음으로 간주해 버림
`max_speech_ms`	15000	멈춤 없이 이어지는 구간을 강제로 끊는 한계
`start_speech_frames`	3	캡처를 시작하는 데 필요한 연속 음성 프레임 수
`pre_roll_ms`	200	발화 시작 직전 오디오를 함께 보관해 첫 음절이 잘리지 않게 함
`vad_aggressiveness`	2	webrtcvad 민감도 0~3 (높을수록 비음성 판정이 공격적)

Gemini 설정

설정	방법
API 키	`export GEMINI_API_KEY=...` 또는 `ASRCorrector(api_key="...")`
여러 키 (순환)	`export GEMINI_API_KEYS=key1,key2,key3` 또는 `api_keys=[...]`. 요청이 키들 사이에서 순환하며, 사용량 제한(429)에 걸리거나 실패한 키는 자동으로 건너뜁니다.
모델	기본값 `gemini-2.5-flash-lite`. `export GEMINI_MODEL=...` 또는 `ASRCorrector(gemini_model="...")` 로 변경
사용자 지시문	`ASRCorrector(prompt="...")` 또는 `process(..., prompt="...")`. 라이브러리가 지시문 뒤에 원본 전사를 붙여 보냅니다.

> **Gemini가 실패해도**(잘못된 키, 사용량 소진, 네트워크 없음) 결과는 반환됩니다. `corrected_text` 에는 여분의 공백만 정리한 원본 전사가 들어가고 `correction_ok` 는 `False` 가 되므로, 교정이 중요하다면 이 값을 확인하세요. 대신 예외를 받고 싶다면 `GeminiCorrector(fallback=False)` 를 사용하세요.

모델을 가져오는 위치

해석 우선순위: 명시적 경로 → Hugging Face 저장소 → `CTC_MODEL_DIR` / `CTC_VOCAB_PATH` → 로컬 sibling 체크아웃 (원래 서버에서만).

소스	방법
Hugging Face Hub	`ASRCorrector(hf_repo="fluffy16/ctc-asr-ko")` 또는 `export CTC_HF_REPO=fluffy16/ctc-asr-ko`
로컬 파일	`ASRCorrector("/path/checkpoint", "/path/vocab.json")` 또는 `CTC_MODEL_DIR` / `CTC_VOCAB_PATH`
버전 고정	`ASRCorrector(hf_repo="fluffy16/ctc-asr-ko", hf_revision="<commit-sha>")`
코드에서 토큰 전달	`ASRCorrector(hf_repo=..., hf_token="hf_...")` (`huggingface-cli login` 대신 사용 가능)

성능 (샘플 세트에서 측정)

모델이 이미 로드된 상태에서 음성 인식 호출만 측정한 시간입니다. Gemini 교정은 네트워크에 따라 호출당 약 0.5~1초가 추가됩니다.

오디오 길이	GPU (CUDA)	CPU
2 초	0.012 초	0.11 초
5 초	0.025 초	0.20 초
14 초	0.026 초	0.62 초
22 초	0.034 초	0.83 초

- **GPU**: 클립 길이와 관계없이 약 25 ms.
- **CPU**: 오디오 1초당 약 0.04초로 길이에 비례합니다. 실시간 스트리밍에도 충분합니다.
- **모델 로드**: 객체를 만들 때 한 번, 약 0.4초.
- **Hugging Face 최초 다운로드**: 약 1.2 GB, 이후 캐시가 유지됩니다.
- 실제로는 음성 인식보다 Gemini 호출이 전체 시간의 대부분을 차지합니다.

제약 사항

- 교정은 AI가 문장을 다시 쓰는 방식입니다. 어미나 시제를 바꾸거나 문장 부호를 추가할 수 있고, 가끔 잘못 추측하기도 합니다. 음성 인식 원문 그대로가 필요하면 `r.transcript` 를 보관하세요.

- CTC 모델은 들리는 소리를 음절 단위로 그대로 적기 때문에(예: `앉았어요`를 `안자쩌요`로), 원본 출력은 표준 맞춤법과 다른 경우가 많습니다. 교정 단계가 이를 바로잡습니다.
- 모델은 발화 전체를 전사합니다. 단어 단위 실시간 출력은 없으며, "실시간"은 멈춤 직후 전사가 나타난다는 의미입니다.
- 침묵 기준으로 구간을 나누므로 긴 문장이 멈춤 지점에서 나뉘어, 문장 전체가 아니라 조각 단위로 교정될 수 있습니다.
- 하나의 `ASRCorrector`는 한 번에 하나의 전사만 수행합니다. 병렬 처리가 필요하면 워커 프로세스마다 인스턴스를 하나씩 두세요.
- 모델은 한국어 전용이며, 마비말·비정형 발화에 맞게 파인튜닝되어 있습니다.
- 교정할 때마다 전사 텍스트가 Google Gemini API로 전송됩니다. 다루는 데이터에 대해 이것이 허용되는지 확인하세요.

업데이트

시점	관리자 작업	사용자 작업
라이브러리 코드 변경	GitHub 저장소에 `git push`	`pip install --upgrade "ctc-asr-text-correct[hub] @ git+https://github.com/Aditi1612/ctc_asr_text_correct_library.git"`
모델 체크포인트 변경	`examples/upload_to_hf.py` 재실행	없음 — 다음 실행 시 새 버전을 받음 (`hf_revision`을 고정하지 않은 경우)

문제 해결

증상	해결
다운로드 시 `GatedRepoError` 또는 `401`	접근이 아직 승인되지 않았거나 토큰이 설정되지 않았습니다. 모델 페이지를 확인한 뒤 `huggingface-cli login` 을 실행하거나 `HF_TOKEN` 을 설정하세요.
`No checkpoint found`	`hf_repo=` / `model_dir=` 를 전달하거나 `CTC_HF_REPO` 를 설정
`No Gemini API key found`	`GEMINI_API_KEY` 를 설정하거나 `api_key=` 를 전달. 음성 인식만 쓸 거라면 `Transcriber` 를 사용
`correction_ok` 가 `False`	Gemini가 실패했습니다. 로그의 경고(잘못된 키, 사용량, 네트워크)를 확인하세요. 대신 원본 전사가 반환되었습니다.
`Hugging Face Hub support needs 'huggingface_hub'`	`[hub]` extra로 설치
VAD 백엔드가 `energy` 이고 검출이 거침	`pip install webrtcvad`
mp3 / m4a 파일 디코딩 실패	시스템에 ffmpeg 설치
CUDA out of memory	`ASRCorrector(..., force_cpu=True)` 사용 — CTC 모델은 CPU에서도 빠름